

UNIT STUDY NOTES

Built-in Data Structures and Functions

Lists, tuples, sets and dictionaries are the containers you will use in essentially every line of data work. This unit covers their semantics, the cost of their operations, and how function definitions scope and pass data.

 3 source notebooks

 145 topic sections extracted

 8 study sections

Unit Overview

Welcome to Session 4 of the Python track. In this session you will build a complete, working mental model of Python lists — the single most used data structure in everyday Python code. Whether you are storing the marks of fifty students, the rows of a spreadsheet, or the words of a document, lists are usually the container you reach for first. Getting genuinely comfortable with them is not optional: almost every data-science pipeline, every script that reads user input, and every algorithm you will write later in this course touches a list in some way.

A list is a data type under which you can store multiple items under one name. Internally it behaves like a **dynamic array**: it can grow and shrink automatically as you add or remove items, which is why a list can hold one item at the start and five thousand later without you doing anything special.

Here is the roadmap for this session:

1. **What lists are** — the definition and why a single variable is not enough.
2. **Lists vs Arrays** — how a Python list differs from a fixed-size C/Java array, and how lists are stored in memory.
3. **The 8 characteristics** — ordered, mutable, heterogeneous, duplicates allowed, dynamic, nestable, indexable, and able to hold any Python object.
4. **Creating lists** — the `[]` literal, empty lists, 2D/3D nesting, and the `list()` type-conversion constructor.
5. **Accessing items** — positive and negative indexing, slicing with `start:stop:step`, and indexing into nested lists.
6. **Editing items** — index assignment and the surprising power of slice assignment.
7. **Deleting items** — `del`, `remove`, `pop`, and `clear`, and when each one is the right tool.
8. **Operations** — concatenation with `+`, repetition with `*`, membership with `in / not in`, looping, and equality.
9. **Functions on lists** — `len`, `min`, `max`, `sorted` and the in-place methods `sort`, `reverse`, `count`, `index`, plus `append`, `extend`, `insert`.

Before diving in, keep two big ideas in mind. First, a list is **mutable** — you can change it in place, exactly like you will later see tuples cannot be changed. Second, a list stores **references** to objects, not the objects themselves; that single fact explains most of the "weird" list behaviour you will meet, including how copies

behave. Master these two ideas and everything else in the session follows naturally. Let us start with the basics: what lists are, and why they are required in programming.

Welcome to Session 5. In the previous session you mastered the list — a mutable, ordered workhorse. Now we study the other three built-in containers that Python gives you out of the box: **tuples**, **sets**, and **dictionaries**. Each one answers a different question, and knowing which container to reach for is a core Python skill.

- A **tuple** is an immutable list. You create a fixed record — coordinates, a date, a student's basic identity, a function's multiple return values — and once created, it cannot change. This immutability makes tuples hashable, which lets them serve as dictionary keys and set elements (things a list can never be).
- A **set** is an unordered collection of unique, hashable items. It is conceptually a mathematical set: elements have no duplicates and no positions, and you can perform union, intersection, difference, and symmetric difference directly with operators like `|`, `&`, `-`, and `^`. Set membership checks are $O(1)$, which makes sets the tool of choice for "is it there?", deduplication, and removing overlapping items from data.
- A **dictionary** stores key→value pairs, like a phone book, a map, or a JSON object. Keys must be unique and immutable; values can be anything, including other dictionaries. Reading a value by its key is $O(1)$, and dictionaries power most of the look-up, grouping, and counting logic you will write.

The session is organised in three large blocks, each following the same rhythm as Session 4: characteristics → creation → accessing → editing/adding/deleting → operations → functions → comprehension.

1. **Tuples:** creating tuples (mind the comma!), accessing and slicing, why you cannot edit them, the `+` / `*` operations, the two methods `count` and `index`, list-vs-tuple trade-offs, and unpacking.
2. **Sets:** creating sets, why you use `set()` and not `{}` for empty, no-index access, adding with `add` / `update`, deleting with `remove` / `discard` / `pop` / `clear`, the four operators, methods like `union` / `intersection_update` / `issubset`, frozensets, and set comprehension.
3. **Dictionaries:** creating dicts, reading with `[]` vs `get()`, adding/editing/removing pairs, membership and iteration, functions like `len` / `sorted` / `update`, and dictionary comprehension.

Keep one comparison running through the whole session as a thread: **list (mutable, ordered, slower, heavy)**, **tuple (immutable, ordered, faster, light)**, **set (mutable, unordered, unique)**, **dict (mutable, keyed)**. Everything you read in this session is a variation on that table. Let us begin with tuples.

Welcome to Session 6. Up to now you have written *scripts* — flat sequences of statements. In this session you learn to write **functions**: named, reusable blocks of logic that take inputs, do work, and hand back a result. Functions are the single most important organisational tool in Python. Every program of more than a few lines is built out of functions, every library you import is a bundle of functions, and the standard Python functions like `len`, `print`, and `map` are the models you will imitate.

A function gives you three things at once:

1. **Reusability** — write the tricky logic once, call it a thousand times. The same block that computes loan interest today is used unchanged in next month's script.
2. **Abstraction** — callers only need to know the inputs and the output, not the internals. That separation of *what* from *how* is why huge codebases stay manageable.

3. **Modularity & readability** — a program reads top-down as a list of named actions (`load_data(...)`, `clean_data(...)`, `train_model(...)`) instead of one long wall of statements.

Here is the roadmap for this session:

1. **Anatomy of a function** — `def`, parameters, indented body, docstring, and `return`. What makes a call work.
2. **Two points of view** — the author who defines the function versus the user who only calls it.
3. **Arguments** — default, positional, and keyword arguments, and the rules for mixing them.
4. **`*args` and `**kwargs`** — collecting a variable number of positional or keyword arguments.
5. **`return` and `None`** — what a function hands back, and the special "no value" that makes bugs when ignored.
6. **Variable scope** — global vs local names, why assigning inside a function shadows the outside, and `UnboundLocalError`.
7. **Nested functions & recursion danger** — functions defined inside functions, and the crash that comes when recursion never stops.
8. **First-class citizens** — functions are objects: assignable, storable, passable, returnable.
9. **Lambda functions** — tiny anonymous one-expression functions.
10. **Higher-order functions** — `map`, `filter`, `reduce`, which take functions as arguments.

Through every topic runs one idea: **a function is a value with behaviour**. Once that clicks, everything else — passing functions around, lambdas, and `map` / `filter` / `reduce` — feels natural. Let us start by building the very first function and dissecting it line by line.

Prerequisites

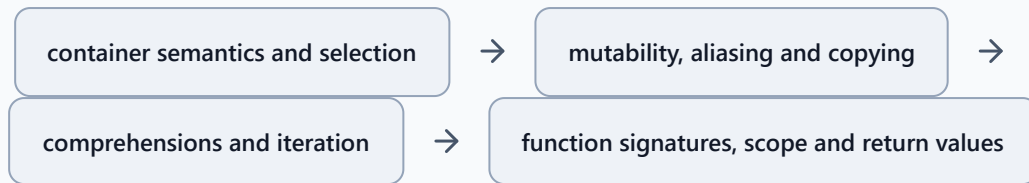
→ Python Core Fundamentals

What you will be able to do

- ✓ Choose between list, tuple, set and dict from the access pattern you need
- ✓ Reason about mutability and aliasing when passing data around
- ✓ Write functions with clear parameters, defaults and return contracts
- ✓ Avoid the classic list/tuple aliasing and mutable-default bugs

What This Unit Covers

CONCEPT MAP



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Lists in Python

What are Lists

List is a data type where you can store multiple items under 1 name. More technically, lists act like dynamic arrays which means you can add more items on the fly.

`L = [ 20, 'Jessa', 35.75, [30, 60, 90] ]`

Diagram illustrating list indexing:

- `L[0]` points to `20`
- `L[1]` points to `'Jessa'`
- `L[2]` points to `35.75`
- `L[3]` points to `[30, 60, 90]`

- Why Lists are required in programming?

Array Vs Lists

- Fixed Vs Dynamic Size
- Convenience -> Hetrogeneous
- Speed of Execution
- Memory

Theory : Lists vs Arrays & how memory works

Array (C / Java style) :

- Fixed size - you decide the capacity BEFORE filling it. Adding one more element when it is full needs a whole new array.
- Homogeneous - every slot must hold the same type.
- Very fast & very compact - values sit side by side in one memory block.

Python List (dynamic array) :

- **Dynamic** - grows and shrinks automatically as you add/remove items. Python reserves a little extra room and reclaims it as needed.
- **Heterogeneous** - one list can hold an int, a float, a string, a bool and even another list.
- A little slower and uses more memory than a raw C array (because of that extra coating).

How a list is actually stored : A list does NOT copy the numbers inside itself. It keeps a contiguous block of **references (pointers)** to objects that live elsewhere in memory. `id(L)` is the address of the list, `id(L[0])` is the address of the integer object it points to.

Why do we even need lists? - A single integer variable can store only ONE value. Lists let one name manage a whole collection, so you can process 100 students without writing 100 variables.

PYTHON

```
L = [1, 2, 3]
print(id(L))      # ONE unique id for the list object itself
print(id(L[0]))   # the SAME id as id(1) - Python caches small ints
```

Deep Dive : Lists vs Arrays & memory

- A list is a **collection of references** — every item is a pointer to an object stored elsewhere.
- `id()` returns an object's unique memory address (in CPython).
- The list's address and its items' addresses are all different; Python caches small integers, so `id(L[0])` matches `id(1)`.
- Internally a Python list is an array of `PyObject*` pointers (~8 bytes per pointer).
- **Array (C/Java style)**: same-type, contiguous, fixed size, fast. **List (Python)**: heterogeneous, dynamic size, flexible.
- Nested containers are **not** flattened — a list inside a list stays a separate object.
- Indexing is $O(1)$, but each item carries object overhead, so lists are slower and heavier than `numpy` arrays.
- `id(L)` stays constant while items are added; the internal pointer array may be resized, but the list object itself keeps its identity.

PYTHON

```
L = [1,2,3]

print(id(L))
print(id(L[0]))
print(id(L[1]))
print(id(L[2]))
print(id(1))
print(id(2))
print(id(3))
```

PYTHON

```
L = [1, 2, 3]
print(id(L))      # address of the List object itself
print(id(L[0]))   # address of the integer 1 it points to
```

Characterstics of a List

- Ordered
- Changeble/Mutable
- Hetrogeneous
- Can have duplicates
- are dynamic
- can be nested
- items can be accessed
- can contain any kind of objects in python

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Theory : The 8 characteristics - one by one

1. **Ordered** - items keep the order you inserted them. Order is part of the value : `[1,2,3] == [3,2,1]` is `False`.
2. **Mutable / Changeable** - you can append, edit, insert, delete items in place. (Strings & tuples cannot.)
3. **Heterogeneous** - mixed types in one list : `[1, True, 5.6, 'Hello']`.
4. **Duplicates allowed** - `[1,1,2]` is perfectly fine.
5. **Dynamic** - it stretches and shrinks automatically at runtime.
6. **Nestable** - lists inside lists (2D, 3D ... as deep as you like).
7. **Accessible** - each item reachable by its position (index).
8. **Can hold ANY Python object** - even functions : `[print, type, input]` is legal!

Deep Dive : Characteristics of a list

Characteristics:

1. **Ordered** — insertion order preserved; equality compares order
 2. **Mutable** — add/remove/change in place
 3. **Homogeneous/heterogeneous** — mixed types allowed
 4. **Indexable** — `L[i]`
 5. **Slicable** — `L[start:stop:step]`
 6. **Allows duplicates** — the same value can appear many times
 7. **Dynamic** — size scales up and down automatically
 8. **Iterable** — for loop, comprehension, membership
- Equality = element-wise + order
 - `[1,2,3,1] == [3,2,1]` → False (order)
 - Sets have no order and no duplicates — that is the defining difference of a list

PYTHON

```
L = [1,2,3,1]
L1 = [3,2,1]

L == L1
```

PYTHON

```
L = [1,2,3,1]
L1 = [3,2,1]
L == L1    # False - ORDER MATTERS
```

Creating a List

Lists: What/Why/Semantics

Mutable ordered sequences. In-place ops; assignment creates alias.

Common mistakes

Shallow vs deep copy confusion.

Theory : Creating lists — every legal way

Via the `list()` constructor (type conversion):

- `[]` is the fast literal syntax for an empty list.
- `list(iterable)` converts any iterable (string, tuple, range, set) into a list. It simply walks the iterable and collects its items.

- Be careful: `list('hello')` splits the string into characters because a string iterates as its characters. To wrap a string as one item, use `['hello']` instead.
- The literal syntax is the idiomatic choice in ~99% of cases; the constructor shines when you are converting a `range`, tuple, or set into a list.

PYTHON

```
[ ]                # empty list
[1, 2, 3, 4, 5]    # 1D homogeneous
[1, 2, 3, [4, 5]]   # 2D (a list inside a list)
[[[1,2],[3,4]],[[5,6],[7,8]]] # 3D
[1, True, 5.6, 5+6j, 'Hello'] # heterogeneous
```

PYTHON

```
list('hello')      # -> ['h','e','l','l','o'] (iterates the string)
list(range(5))     # -> [0,1,2,3,4]
list((1,2,3))      # tuple -> list
```

Deep Dive : Creating lists

- **Dimension/nesting** — a list inside a list
- 2D = list of lists (matrix), 3D = tensor style
- Constructor: `list(range(5))`, `list('abc')` → chars split
- Empty check: `not L` vs `len(L)==0`
- Mutable default: `list()` gives a new object
- Key point for 2D: `L[0][1]` nested index

PYTHON

```
# Empty
print([ ])
# 1D -> Homo
print([1,2,3,4,5])
# 2D
print([1,2,3,[4,5]])
# 3D
print([[[1,2],[3,4]],[[5,6],[7,8]]])
# Hetrogenous
print([1,True,5.6,5+6j,'Hello'])
# Using Type conversion
print(list('hello'))
```

PYTHON

```
[ ] # empty
[1,2,3,4,5] # 1D list
[1,2,3,[4,5]] # 2D list
[[[1,2],[3,4]],[[5,6],[7,8]]] # 3D list
list('hello') # ['h','e','l','l','o']
range(5) # iterable → List(range(5))
```

Theory : Indexing & Slicing

Indexing = grab ONE item by position.

- Positive index counts from the LEFT starting at 0 : `L[0]` , `L[1]` ...
- Negative index counts from the RIGHT starting at -1 : `L[-1]` is the last item.
- Stepping out of range raises `IndexError: list index out of range` .

Slicing = grab a RANGE : `L[start : stop : step]`

- `start` inclusive, `stop` EXCLUSIVE, `step` is the jump.
- Slicing always returns a NEW list - the original is untouched.

Nested lists chain `[]` : for `L = [[1,2],[3,4]]` , `L[1][0]` -> 3.

PYTHON

```
L = [1, 2, 3, 4, 5, 6]
L[1:4] # -> [2, 3, 4]
L[:3] # from the start -> [1, 2, 3]
L[::2] # every 2nd -> [1, 3, 5]
L[::-1] # REVERSE the list -> [6, 5, 4, 3, 2, 1]
```

Deep Dive : Indexing & slicing lists

- Positive 0..n-1 | negative -1..-n
- Nested indexing — step by step outer→inner
- Slicing: returns **new list** (shallow copy of refs)
- Slice on nested → shallow — inner lists still SAME objects
- Out of range index → `IndexError`; slice → clamp
- `.copy()` == `L[:]`
- Negative slice: `L[-2:]` last two

PYTHON

```
# Indexing
L = [[1,2],[3,4]],[5,6],[7,8]]
#positive
#print(L[0][0][1])

# Slicing
L = [1,2,3,4,5,6]

print(L[::-1])
```

PYTHON

```
L = [[1,2],[3,4]],[5,6],[7,8]]
L[0]      # [[1,2],[3,4]]
L[0][1]   # [3,4]
L[0][1][0] # 3
L[-1]     # [[5,6],[7,8]]
```

Theory : append vs extend vs insert

- `append(x)` - adds ONE value at the end. The value becomes a single element, even if it is a list.
- `extend(iterable)` - takes an iterable and adds each of its items one by one. Strings get split into characters!
- `insert(index, value)` - pushes a value at any position, shifting the rest right.

Memory hook : append adds ONE box, extend dumps the contents of the box into the list.

PYTHON

```
L = [1, 2, 3, 4, 5]
L.append([6,7,8])    # -> [1, 2, 3, 4, 5, [6, 7, 8]]    (nested!)
L.append(9)          # -> [1, 2, 3, 4, 5, [6, 7, 8], 9]
L.extend([6,7,8])    # -> [1, 2, 3, 4, 5, 6, 7, 8]      (flat merge)
L.extend('delhi')    # -> [..., 'd','e','l','h','i']
L.insert(1, 100)     # -> [1, 100, 2, 3, ...]
```

Deep Dive : append vs extend vs insert

- `append(x)` — x as a SINGLE object (a list becomes nested)
- `extend(iterable)` — items unpacked one by one
- `insert(i,x)` — shifts the rest right
- All mutate in place, return None
- extend any iterable → costs time proportional to size

PYTHON

```
# append
L = [1,2,3,4,5]
L.append(True)
print(L)
```

PYTHON

```
# extend
L = [1,2,3,4,5]
L.extend([6,7,8])
print(L)
```

Theory : Editing items - in place

Lists are MUTABLE, so single items and whole slices can be changed directly.

Slice assignment can even change the LENGTH : This is exactly what strings and tuples CANNOT do - that difference is the heart of "mutable vs immutable".

PYTHON

```
L = [1, 2, 3, 4, 5]
L[-1] = 500      # edit by (negative) index -> [1, 2, 3, 4, 500]
L[1:4] = [200,300,400] # slice assignment -> [1, 200, 300, 400, 500]
```

PYTHON

```
L[1:3] = [7, 8, 9, 10] # replaces 2 items with 4 -> list grows
L[1:4] = []            # deletes a slice
```

Deep Dive : Editing list items — in place

- Lists are MUTABLE → in-place editing is allowed (strings could not do this!)
- Index assignment: `L[i] = value`
- Slice assignment: `L[start:stop] = [..]` — replace a subrange
- `L[:] = [..]` → replace all (fast clear/refill)
- Contrast: strings immutable → `TypeError`; lists mutable
- Direct; no copy required

PYTHON

```
L = [1,2,3,4,5]

# editing with indexing
L[-1] = 500

# editing with slicing
L[1:4] = [200,300,400]

print(L)
```

PYTHON

```
L = [1,2,3,4,5]
L[-1] = 500      # index assignment → [1,2,3,4,500]
L[1:3] = [20,30] # slice assignment → replace range
```

Theory : Deleting from a list

Four tools, two mindsets :

By position (index based)

- `del L[i]` - delete the item at index i.
- `del L[a:b]` - delete a whole slice.
- `del L` - delete the VARIABLE itself (using it later -> `NameError`).

By value (value based)

- `L.remove(value)` - deletes the FIRST occurrence. If the value is absent -> `ValueError`.
- `L.pop()` - removes AND returns the last item (like a stack / LIFO).
- `L.pop(i)` - removes AND returns the item at index i.
- `L.clear()` - empties the whole list -> `[]`.

`pop()` gives you the value back, `del` and `remove` do not.

PYTHON

```
L = [1,2,3,4,5]
del L[-1]      # [1,2,3,4]
L.pop()        # 4 (and L becomes [1,2,3])
L.remove(2)    # [1,3]
L.clear()      # []
```

Deep Dive : Deleting from list — 4 tools

Comparison:

- `del` — statement; index/slice; returns nothing
- `remove(x)` — first occurrence by value; missing → `ValueError`

- `pop(index)` — index or last; returns the value
- `clear()` — empty the list
- `del L` deletes whole list variable (vs `del item`)
- Memory: `pop` removes reference

PYTHON

```
# del
L = [1,2,3,4,5]

# indexing
del L[-1]

# slicing
del L[1:3]
print(L)
```

PYTHON

```
# remove

L = [1,2,3,4,5]

L.remove(5)

print(L)
```

Operations on Lists

- Arithmetic
- Membership
- Loop

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Theory : Operations on lists

Arithmetic

- `+` concatenates two lists -> a NEW list (originals untouched) : `[1,2] + [3,4] = [1,2,3,4]`
- `*` repeats the list : `[1,2] * 3 = [1,2,1,2,1,2]`

Membership

- `in` / `not in` -> is a value present? Works at ANY depth for nested lists : `[5,6] in [[5,6],7]` -> True.

Looping - `for i in L` walks the items in order. For `[[[1,2],[3,4]],[[5,6],[7,8]]]`, each `i` is one 2D sub-list.

Equality - `==` compares items one by one in order : `[1,2,3] == [3,2,1]` -> False (order matters).

Deep Dive : List operations

- `+` — concatenate (new object, $O(\text{len})$)
- `*` — repeat n times (new object)
- `in` — membership; checks an item or a nested list
- Loop over list:
- Nested iteration: for sub in L: for x in sub
- Concatenation vs extend: `+` makes a new list, extend mutates

PYTHON

```
# Arithmetic (+ , *)

L1 = [1,2,3,4]
L2 = [5,6,7,8]

# Concatenation/Merge
print(L1 + L2)
```

PYTHON

```
print(L1*3)
```

Theory : The list toolkit - functions & methods

Built-in functions (return NEW results, never touch the list)

Methods (many work IN PLACE - they change the list and return None)

sort() vs sorted() - the classic trap :

copy() & the aliasing danger : Note : `copy()` is SHALLOW - nested lists are still shared. Use `copy.deepcopy` for full independence.

PYTHON

```
len(L)           # number of items
min(L) / max(L)  # smallest / largest item
sorted(L)        # NEW sorted List (ascending)
sorted(L, reverse=True) # NEW sorted List (descending)
```

PYTHON

```
L.count(x)           # how many times x appears
L.index(x)           # position of FIRST x (ValueError if missing)
L.reverse()          # reverses permanently, in place (returns None)
L.sort()             # sorts in place (returns None)
```

Deep Dive : List functions

- Builtin functions return new: `len/min/max/sorted`
- Methods mutate in place: `sort/reverse/count/index`
- `sorted(L)` vs `L.sort()` — copy vs original
- `sum(L)`, `any(L)`, `all(L)`
- index vs count: find position vs count freq

PYTHON

```
# Len/min/max/sorted
L = [2,1,5,7,0]

print(len(L))
print(min(L))
print(max(L))
print(sorted(L,reverse=True))
```

PYTHON

```
# count
L = [1,2,1,3,4,1,5]
L.count(5)
```

Deep Dive : copy — shallow vs deep

- `copy()` is **shallow**: it creates a new outer list, but the items inside are still the same objects.
- With nested lists the inner lists stay **shared**:
- For a fully independent clone use `import copy; copy.deepcopy(M)` — everything, including nested levels, is copied.
- Compare plain assignment `M3 = M`: here both names refer to the *same* list object, so no copy happens at all.
- `id()` is the reliable way to answer "are these two separate lists?" — a copy differs from the original, an alias does not.
- Nested-copy gotchas are a classic interview topic; remember shallow vs deep and you have cracked the pattern.

PYTHON

```
# copy -> shallow
L = [2,1,5,7,0]
print(L)
print(id(L))
L1 = L.copy()
print(L1)
print(id(L1))
```

PYTHON

```
L = [2, 1, 5, 7, 0]
L1 = L.copy()    # NEW outer list, same inner references
L2 = L[:]        # same as copy
id(L) == id(L1)  # False - separate objects
L1 == L          # True - values equal
```

List Comprehension

List Comprehension provides a concise way of creating lists.

The pattern reads: build a new list by evaluating `expression` for every `item` drawn from `iterable`, keeping only those items that pass the optional `condition`.

Advantages of List Comprehension

- More time-efficient and space-efficient than equivalent `for` loops (the loop runs inside C).
- Require fewer lines of code — a 3-line loop collapses into one readable line.
- Transform an iterative statement into a formula-like expression, which is easier to inspect and reason about.

You will use list comprehension constantly in data work — mapping values, filtering rows, flattening matrices — so the next few cells walk through its anatomy carefully.

PYTHON

```
new_list = [expression for item in iterable if condition]
```

Theory : Comprehension - anatomy & how to read it

The shape : Always THREE parts : the `expression` you compute, the `for` that supplies items, and an optional `if` filter.

Read it like the loop it replaces :

- the expression sits FIRST (where `append()` would be)
- the loop comes second, filters after - exactly the nested order of the original loop.

Stacked `if` s = AND. Multiple `for` s = nested loops : Why prefer it? fewer lines, faster (built-in C loop), and it turns a loop into a formula. Avoid ultra-complex comprehensions - readability wins.

PYTHON

```
newlist = [expression for item in iterable if condition]
```

PYTHON

```
L = []
for i in range(1, 11):
    if i % 5 == 0:
        L.append(i)
# becomes
L = [i for i in range(1, 11) if i % 5 == 0]
```

Deep Dive : List comprehension — anatomy

Shape: `[expression for item in iterable if condition]`

- Order: `for` first, then `if`
- Nested comp = nested for loops
- Read it left-to-right as: "for each ... in ..., if ..."
- Faster & more concise than a loop
- Memory: builds a full list (use a generator expression `(x for x in...)` for lazy evaluation)

PYTHON

```
# Add 1 to 10 numbers to a List
L = []

for i in range(1,11):
    L.append(i)

print(L)
```

PYTHON

```
L = [i for i in range(1,11)]
print(L)
```

2 ways to traverse a list

- itemwise
- indexwise

Theory : Traversing - itemwise vs indexwise

1. Itemwise - read each value directly : Best when you only need the VALUES.

2. Indexwise - need the position, or must EDIT the list while looping : `range(len(L))` hands you valid indexes; `L[i]` fetches the value.

Pythonic middle ground - enumerate() : Choose itemwise for pure reading, indexwise/enumerate when the index itself is part of the job.

PYTHON

```
for i in L:
    print(i)
```

PYTHON

```
for i in range(len(L)):
    print(L[i])      # or L[i] = ... some new value
```

Deep Dive : Traversing lists — 2 ways

- **Itemwise**: use when you only need the value; fast, readable
- **Indexwise**: when you also need the index / want to mutate a position
- `enumerate` — the Pythonic way to get both
- Nested lists: traverse by depth
- When editing by index is required: indexwise is best

PYTHON

```
# itemwise
L = [1,2,3,4]

for i in L:
    print(i)
```

PYTHON

```
# indexwise
L = [1,2,3,4]

for i in range(0,len(L)):
    print(L[i])
```

Zip

The `zip()` function returns a zip object, which is an iterator of tuples where the first item in each passed iterator is paired together, and then the second item in each passed iterator are paired together.

If the passed iterators have different lengths, the iterator with the least items decides the length of the new iterator.

Theory : `zip()` - pairing lists together

`zip(L1, L2)` pairs the items of two (or more) iterables **position by position** into tuples.

Rules :

- `zip` itself returns an ITERATOR - wrap with `list()` / `tuple()` to see it, or loop over it directly.
- If one list is shorter, the **shortest** one decides the length (extra items are dropped).

The super-power - unpacking in a comprehension : This is THE tool when two lists must be processed column by column (temperatures per day, marks per student...).

PYTHON

```
L1 = [1, 2, 3, 4]
L2 = [-1, -2, -3, -4]
list(zip(L1, L2))      # -> [(1, -1), (2, -2), (3, -3), (4, -4)]
```

PYTHON

```
[i + j for i, j in zip(L1, L2)]  # index-wise addition -> [0,0,0,0]
```

Deep Dive : `zip()` — pairing lists

- `zip(L1, L2)` — iterator of tuples, one from each
- Stops at SHORTEST list
- `list(zip(L1,L2))` → [(1,-1),(2,-2),...]
- `dict(zip(keys, values))` — keys:values mapping
- Parallel iteration standard tool
- Indexwise operation without index variable
- Item L[print]: elementwise ops

PYTHON

```
# Write a program to add items of 2 Lists indexwise
```

```
L1 = [1,2,3,4]
L2 = [-1,-2,-3,-4]

list(zip(L1,L2))

[i+j for i,j in zip(L1,L2)]
```

PYTHON

```
L = [1,2,print,type,input]

print(L)
```

Disadvantages of Python Lists

- Slow
- Risky usage
- eats up more memory

PYTHON

```

a = [1,2,3]
b = a.copy()

print(a)
print(b)

a.append(4)
print(a)
print(b)

# Lists are mutable

```

Theory : Strategy for the list programs

1. Read the problem, decide what you NEED from the source list (order? uniqueness? positions?).
2. Pick the tool :
 - **Order / positions** -> indexing, slicing, indexwise loops
 - **No duplicates** -> think `set` | **counting** -> think `dict`
 - **Transform each item** -> comprehension / map | **User data** -> `input().split()`
3. Plan the 7 exercises with these hints :
 - odd/even split -> loop + `% 2` + two accumulators
 - list input from user -> `input().split()` then cast each with `map(int, ...)`
 - merge without `+` -> use `extend`
 - replace an item -> find `index()`, then assign at that index
 - 2D list -> 1D -> for each row, for each cell, append
 - remove duplicates -> build a `seen` check (or `set` when order is not sacred)
 - ascending check -> compare with `sorted(L)`

Deep Dive : List programs — strategy

Problem: even/odd split

General strategy:

1. Understand input/output
 2. Initialize empty result lists
 3. Iterate, filter/classify
 4. Append to right target
- Comprehension version: `[x for x in L if x%2==0]`
 - Source: same for every list program

PYTHON

```
# Create 2 Lists from a given List where
# 1st list will contain all the odd numbers from the original list and
# the 2nd one will contain all the even numbers

L = [1,2,3,4,5,6]
```

PYTHON

```
L = [1,2,3,4,5,6,7,8]
evens = []
odds = []
for x in L:
    if x % 2 == 0:
        evens.append(x)
    else:
        odds.append(x)
```

Deep Dive : List input from user

- `input().split()` → strings list
- `map(int, ...)` → int objects
- `list(map(...))` → ints list
- multiples: comprehension `[int(x) for x in input().split()]`
- User input coverage common pattern

PYTHON

```
# Line separated
n = int(input('how many : '))
L = []
for _ in range(n):
    L.append(input('enter : '))
```

PYTHON

```
# space separated - map int
L = list(map(int, input('enter space separated : ').split()))
# comma separated
L = input().split(',')
```

Deep Dive : Manual merge (no +)

Or extend:

- Avoid mutating original lists
- Loop copying both
- `L1 + L2` builtin — but practice manual
- `extend` or `chain` — no +

- result ≠ reference to L1/L2

PYTHON

```
# Write a program to merge 2 list without using the + operator
L1 = [1,2,3,4]
L2 = [5,6,7,8]
```

PYTHON

```
L1 = [1,2,3,4]
L2 = [5,6,7,8]
merged = []
for x in L1:
    merged.append(x)
for x in L2:
    merged.append(x)
```

Deep Dive : Replace item in list

- Indexwise loop (edit in place)
- Replace ALL occurrences (no break)
- Single: `i = L.index(target); L[i]=new` with try
- In place mutation — list mutable
- String replace not allowed — lists allow
- Extended: `[new if x==target else x for x in L]`

PYTHON

```
# Write a program to replace an item with a different item if found in the list
L = [1,2,3,4,5,3]
# replace 3 with 300
```

PYTHON

```
L = [10, 20, 30, 20, 40]
target = 20
new = 99
for i in range(len(L)):
    if L[i] == target:
        L[i] = new
# L → [10,99,30,99,40]
```

Deep Dive : Flatten 2D → 1D

Comprehension:

- Nested iteration row→item
- Comprehension order = for nesting order
- Term: flattening

- Deep nesting: recursion/generic
- Images, matrices data prep common

PYTHON

```
M = [[1,2],[3,4],[5,6]]
flat = []
for row in M:
    for item in row:
        flat.append(item)
# [1,2,3,4,5,6]
```

PYTHON

```
flat = [item for row in M for item in row]
```

Deep Dive : Remove duplicates

Set based:

- Manual: check-then-append, $O(n^2)$, preserves order
- set: fast but unordered
- dict.fromkeys: order + unique (dict insertion order)
- Interview favorite — know all 3
- Stability (order) trade-off

PYTHON

```
# Write a program to remove duplicate items from a list
```

```
L = [1,2,1,2,3,4,5,3,4]
```

PYTHON

```
L = [1,2,1,2,3,4,5,3,4]
unique = []
for x in L:
    if x not in unique:
        unique.append(x)
# [1,2,3,4,5] — IN ORDER
```

Deep Dive : Check sorted/ascending

- Adjacent comparison: `L[i] > L[i+1]` violates
- One pass $O(n)$
- break early — efficient
- Pythonic: `all(L[i] <= L[i+1] for i in range(len(L)-1))`
- Or `L == sorted(L)`

- Edge: empty/single → True

PYTHON

```
L = [1,2,3,4,5]
asc = True
for i in range(len(L)-1):
    if L[i] > L[i+1]:
        asc = False
        break
print(asc)
```

Summary

In this session you learned that a list is a mutable, ordered container that stores references to objects and behaves like a dynamic array. You compared lists with fixed-size arrays, listed the eight characteristics of a list, and explored creation, indexing, slicing, editing, and deletion. You then covered the operations — concatenation, repetition, membership, looping, and equality — and finished with the function/method toolkit: `len`, `min`, `max`, `sorted`, `sort`, `reverse`, `count`, `index`, and the three adders `append`, `extend`, `insert`. Two ideas tie it all together: lists are **mutable** (editing in place is allowed), and a list stores **references**, which is why copying has shallow and deep flavours. Wherever you go next — tuples, sets, dictionaries, functions, or algorithms — lists will be your most-travelled companion, and this session has given you the vocabulary to read and write list code fluently.

This session took you through the three remaining built-in containers. You learned that **tuples** are immutable, ordered records created with parentheses and a comma — convenient for fixed data, functions returning multiple values, and (because they are hashable) dict keys and set elements. Tuples carry only `count` and `index`, are lighter and faster than lists, and can be unpacked in one line. You then met **sets** — unordered, duplicate-free, mutable collections whose fast O(1) membership and mathematical operators (`|`, `&`, `-`, `^`) make them perfect for deduplication and set comparisons; their strict vs tolerant deletions (`remove` vs `discard`), the methods-versus-operators pattern, and immutable `frozenset`. Finally, **dictionaries** store key→value pairs with O(1) lookups: unique hashable keys, values of any type, safe reads with `get()`, assignment that inserts or updates, membership on keys, `items()/keys()/values()` views, `update()` merging, and dictionary comprehension. The running theme is choosing the right container: ordered-and-mutable → list, ordered-and-fixed → tuple, unique-and-membership → set, keyed-lookup → dict.

In this session you built complete functions and understood how they work under the hood. You learned the anatomy — `def`, name, parameters, indented body, docstring, and `return` — and the two points of view of an author and a user. You mastered argument passing: defaults, positional, keyword, and the variable-length `*args` / `**kwargs`. You saw that a function without `return` silently returns `None`, a frequent bug source. Scope taught you that names inside a function are local and that assigning anywhere in a function makes Python treat the name as local (hence `UnboundLocalError`); the LEGB rule (Local → Enclosing → Global → Builtin) summarises name resolution. Nested functions introduced inner helpers and the recursion danger of missing base cases. Functions proved to be first-class citizens — assignable, storable, passable, returnable — which is exactly what makes **lambdas** (tiny anonymous one-expression functions) and the **higher-order functions** `map`, `filter`, and `reduce` work. Write a few small functions each day; that repetition is what turns this anatomy into reflex.

Tuples, Sets & Dictionary

Plan of attack

- Creating a Tuple
- Accessing items
- Editing items
- Adding items
- Deleting items
- Operations on Tuples
- Tuple Functions

Tuples

Immutable ordered sequences - use for fixed collections, hashable if elements hashable (can be dict keys).

Creating Tuples

Tuples

Immutable ordered sequences; hashable if elements hashable.

Theory : Creating tuples - every way

The classic trap : `('hello')` is just a string with brackets around it - parentheses without a comma create no tuple. `('hello',)` is the real single-element tuple.

Why a trailing comma matters : it is the ONLY signal Python uses to know you wanted a tuple and not grouped values.

PYTHON

```
t1 = ()          # empty tuple
t2 = ('hello',)  # ONE item - the trailing comma is MANDATORY!
t3 = (1, 2, 3, 4) # homogeneous
t4 = (1, 2.5, True, [1,2,3]) # heterogeneous (even a list inside)
t5 = (1, 2, 3, (4, 5)) # nested tuple
t6 = tuple('hello') # type conversion -> ('h','e','l','l','o')
```

Deep Dive : Creating tuples — details

- A single-item tuple needs the trailing comma: `(5)` is just the integer `5`, while `(5,)` is a real tuple. The comma — not the parentheses — is what makes a tuple.
- Parentheses are optional in assignment: `t = 1, 2, 3` also creates the tuple `(1, 2, 3)`.
- Create tuples from other iterables: `tuple(range(4))`, `tuple([1, 2])`, `tuple('abc')`.
- A tuple is immutable, but it can *contain* mutable objects such as lists: `t = (1, (2,3), [4,5])` is legal. The list inside is still mutable.

- A tuple is hashable only when all its items are hashable — the reason `(1,2)` can be a dict key but `[1,2]` cannot.

PYTHON

```
# empty
t1 = ()
print(t1)
# create a tuple with a single item
t2 = ('hello',)
print(t2)
print(type(t2))
# homo
t3 = (1,2,3,4)
print(t3)
# hetro
t4 = (1,2.5,True,[1,2,3])
print(t4)
# tuple
t5 = (1,2,3,(4,5))
print(t5)
# using type conversion
t6 = tuple('hello')
print(t6)
```

PYTHON

```
t1 = ()           # empty tuple
t2 = (5,)         # single item – the trailing comma is REQUIRED
t = (1,2,3,4)     # multiple items
tuple('hello')    # ('h','e','l','l','o')
```

Accessing Items

- Indexing
- Slicing

Theory : Accessing tuples - indexing, slicing, nesting

Indexing - exactly like lists :

Slicing - also like lists, but the result is a TUPLE :

Nested access - chain the brackets : Everything that reads a tuple is allowed - everything that changes it is not.

PYTHON

```
t3[0]    # first item
t3[-1]   # last item
```

PYTHON

```
t = (1,2,3,4,5)
t[1:3]      # (2, 3)
t[-1:-4:-1] # (5, 4, 3) - reverse step walks backwards
```

Deep Dive : Accessing tuples

- Indexing/slicing SAME as lists
- Negative indices work
- Slice returns new tuple
- Membership: `3 in t3`
- Iteration: `for x in t3`
- Ordered → index position valid

PYTHON

```
print(t3)
print(t3[0])
print(t3[-1])
```

PYTHON

```
t5[-1][0]
```

Theory : Immutability - you cannot edit or partially delete a tuple

Try to change an item -> **TypeError** : Try to delete one item -> **TypeError** again : You CAN delete the whole variable (now it does not exist) :

Immutable means safe :

- A tuple can be stored in a set / used as a dict key (a list cannot).
- Its contents can never change under you -> fewer bugs.
- It is faster and smaller than a list.

Aliasing subtlety - what "+" really does : With a LIST, `a.append(4)` would change the object b also sees.

PYTHON

```
t3[0] = 100 # TypeError: 'tuple' object does not support item assignment
```

PYTHON

```
del t5[-1] # TypeError: 'tuple' object doesn't support item deletion
```

Deep Dive : Immutability of tuple

- No item assignment, no partial del, no append/extend

- `del t` — variable delete only
- Rebind: `t = t + (6,)` — new object
- Immutable → hashable → dict key / set member
- Contrast: lists mutable; tuple fixed record
- Nested list inside tuple IS mutable (shallow)

PYTHON

```
print(t3)
t3[0] = 100
# immutable just like strings
```

PYTHON

```
t3 = (1,2,3,4,5)
t3[0] = 100      # ✗ TypeError
del t3[-1]       # ✗ TypeError – partial delete not allowed
del t3           # ✔ deletes whole tuple VARIABLE only
```

Adding items

PYTHON

```
print(t3)
# not possible
```

Deleting items

PYTHON

```
print(t3)
del t3
print(t3)
```

PYTHON

```
t = (1,2,3,4,5)

t[-1:-4:-1]
```

Theory : Tuple operations & functions

Same operators as lists, but results are tuples :

Built-ins give results :

A tuple has only TWO methods (because it cannot change) : Lists, by contrast, carry append/extend/insert/remove/pop/sort/reverse too - tuples have no mutation methods because mutation is illegal.

PYTHON

```
(1,2,3) + (4,5,6)      # (1,2,3,4,5,6)    concatenation -> new tuple
(1,2) * 3               # (1,2,1,2,1,2)    repetition
1 in (1,2,3)           # True              membership
for i in (1,2,3): ...  # iteration
```

PYTHON

```
len(t) / sum(t) / min(t) / max(t)
sorted(t, reverse=True) # NOTE: returns a LIST, not a tuple!
```

Deep Dive : Tuple operations & functions

- - concat (NEW tuple), * repeat
- Membership, comparison, iteration — all work
- Sorted(t) → list; len/sum work
- Functions: len/sum/min/max/sorted exist (result list for sorted)
- Operators return new — tuple never mutates
- zip returns tuples — natural tuple usage

PYTHON

```
# + and *
t1 = (1,2,3,4)
t2 = (5,6,7,8)

print(t1 + t2)

print(t1*3)
# membership
1 in t1
# iteration
for i in t1:
    print(i)
```

PYTHON

```
t1 = (1,2,3,4)
t2 = (5,6,7,8)
t1 + t2      # (1,2,3,4,5,6,7,8)
t1 * 2       # (1,2,3,4,1,2,3,4)
3 in t1      # True
tuple(zip(a,b)) # (1,5),(2,6)...
```

Deep Dive : Tuple functions

- The built-in functions `len`, `sum`, `min`, `max` work on tuples exactly as they do on lists.
- `sorted(t)` returns a **new sorted list** — the tuple is untouched, and the result is a list, not a tuple. Use `tuple(sorted(t))` if you need the result as a tuple.

- A tuple exposes only **two methods**, because a tuple cannot change:
 - `t.count(x)` — how many times `x` appears (missing → `0`).
 - `t.index(x)` — position of the first occurrence (missing → `ValueError`).
- That is why `t.count(50)` gives `0` and `t.index(50)` raises an error in the cell below: tuple methods never mutate, they only inspect.
- Lists have far more methods (`append`, `extend`, `sort`, `reverse`, ...) precisely because lists are mutable; tuples get by with just the two read-only methods.

PYTHON

```
# Len/sum/min/max/sorted
t = (1,2,3,4)
len(t)

sum(t)

min(t)

max(t)

sorted(t,reverse=True)
```

PYTHON

```
# count
t = (1,2,3,4,5)

t.count(50)
```

Difference between Lists and Tuples

- Syntax
- Mutability
- Speed
- Memory
- Built in functionality
- Error prone
- Usability

Deep Dive : List vs Tuple — speed & memory

ASPECT	LIST	TUPLE
mutable	yes	no
speed	slower	faster
memory	more (over-allocates buffer)	less (exact size)
hashable	no	yes
use	dynamic data	fixed records, keys

- A list is growable, so it over-allocates extra slots to make `append` cheap; that reserve costs memory.
- A tuple has a fixed, exact size — no reserve, less memory, and better cache behaviour.
- Measured in the cells below: iterating a tuple is a bit faster than iterating a list of the same size.
- Aliasing behaves differently: `a=[1,2]; b=a; a.append(3)` makes `b` also `[1,2,3]` (shared mutable list), whereas `a=(1,2); b=a; a=a+(3,)` rebinds `a` to a new tuple and `b` stays `(1,2)` — tuples never change in place.

PYTHON

```
import time

L = list(range(100000000))
T = tuple(range(100000000))

start = time.time()
for i in L:
    i*5
print('List time',time.time()-start)

start = time.time()
for i in T:
    i*5
print('Tuple time',time.time()-start)
```

PYTHON

```
import sys

L = list(range(1000))
T = tuple(range(1000))

print('List size',sys.getsizeof(L))
print('Tuple size',sys.getsizeof(T))
```

Theory : Unpacking - parallel assignment, swap & star

Unpacking (destructuring) - split a tuple straight into names : Count must match : `a, b = (1,2,3)` ->

`ValueError: too many values to unpack`.

The famous Python swap - no temp variable needed :

Star unpacking - catch the rest in a list :

Zippping tuples into pairs :

Bonus power : functions can RETURN multiple values by returning a tuple, and callers unpack it in one line :

PYTHON

```
a, b, c = (1, 2, 3) # a=1, b=2, c=3
```

PYTHON

```
a, b = b, a
```

Deep Dive : Unpacking — parallel assignment & star

- Unpacking = spreading tuple values into variables
- Count mismatch → `ValueError`
- Star `*` collects remainder into LIST
- Swap trick — most famous tuple feature
- Parallel assignment common Python idiom
- Function returns tuple → unpack in one line
- `zip` produces an iterable of pairs

PYTHON

```
# tuple unpacking
a,b,c = (1,2,3)
print(a,b,c)
```

PYTHON

```
a,b = (1,2,3)
print(a,b)
```

Sets

A set is an unordered collection of items. Every set element is unique (no duplicates) and must be immutable (cannot be changed).

However, a set itself is mutable. We can add or remove items from it.

Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.

Characteristics:

- Unordered
- Mutable
- No Duplicates
- Can't contain mutable data types

Creating Sets

Sets

Unordered collections of unique elements. Useful for membership tests and set operations (union/intersection).

Theory : Sets - unordered, unique, immutable elements

Rules that make a set a set :

1. **Unordered** - there is no position 0, no "first" item.
2. **Unique** - duplicates are removed automatically; `{1,1,2}` is just `{1,2}`.
3. **Elements must be immutable** : ints, floats, strings, tuples are fine. A list inside a set -> `TypeError: unhashable type: 'list'`.
4. The set ITSELF is mutable - you can add and remove items.
5. Membership tests are O(1) (hash table) vs O(n) for lists - huge speed win on big data.

`{1,2,3} == {3,2,1}` is `True` - order plays no part in a set's identity.

PYTHON

```
s = set()           # EMPTY set (<- required; {} is a dict!)
s1 = {1, 2, 3}      # literal
s3 = {1, 'hello', 4.5, (1, 2, 3)} # heterogeneous - but elements must be HASHABLE
s4 = set([1,2,3])   # from a List
s5 = {1,1,2,2,3,3}  # duplicates silently vanish -> {1,2,3}
```

Deep Dive : Creating sets

- Python sets from Python 3.6+ unordered:
 - `s1 = {1,2,3}`, `s2 = {3,2,1}` — **EQUAL**: `s1 == s2` True
 - order irrelevant
- Elements must be **hashable** — lists/sets/dicts not allowed
- Duplicates auto removed: `{1,1,2}` → `{1,2}`
- Empty: must use `set()`

- Set elements unique, fast membership $O(1)$

PYTHON

```
# empty
s = set()
print(s)
print(type(s))

# 1D and 2D
s1 = {1,2,3}
print(s1)
#s2 = {1,2,3,{4,5}}
#print(s2)

# homo and hetro
s3 = {1,'hello',4.5,(1,2,3)}
print(s3)

# using type conversion

s4 = set([1,2,3])
print(s4)
# duplicates not allowed
s5 = {1,1,2,2,3,3}
print(s5)
# set can't have mutable items
s6 = {1,2,[3,4]}
print(s6)
```

PYTHON

```
s1 = {1,2,3}
s2 = {3,2,1}

print(s1 == s2)
```

Deep Dive : Sets are unordered

- No indexing/slicing/ordering — unordered collection
- Ways to access: membership `in`, iteration, `pop()`
- Equality ignores order: `{1,2,3} == {3,2,1}` True
- Unique elements — no duplicates
- Why: hash-table storage, no positional concept
- Access via comprehension/set ops usually

PYTHON

```
s1 = {1,2,3,4}
s1[0:3]
```

PYTHON

```
s1 = {1,2,3,4}
s1[0:3]      # ✗ TypeError – no subscripting
s1[0] = 100   # ✗ no indexing
```

Theory : Because sets are unordered - no indexing, no slicing, no editing

There is simply no "position 0" in a set. Items are scattered by hash.

What this really means :

- You can only READ a set by iterating it (`for i in s`) or asking membership (`x in s`).
- Editing a specific item is impossible - instead you discard the old one and add the new.
- The trade-off is worth it : unordered lookup gives you $O(1)$ "is it there?" checks.

PYTHON

```
s1[0:3]      # TypeError: 'set' object is not subscriptable
s1[0] = 100   # TypeError: 'set' object does not support item assignment
```

Editing Items

PYTHON

```
s1 = {1,2,3,4}
s1[0] = 100
```

Theory : Adding to a set - add vs update

Removing items - two flavours : `remove` = strict (knows you expect it), `discard` = gentle (safe when the item may be absent).

PYTHON

```
S = {1, 2, 3, 4}
S.add(5)           # one item -> {1,2,3,4,5}
S.update([6,7,8])  # many items from any iterable -> {1,...,8}
```

PYTHON

```
S.discard(50)      # no error if missing (gentle)
S.remove(50)       # ValueError if missing (strict)
S.pop()            # removes & returns SOME random item
S.clear()          # empties the set -> set()
del S              # destroys the whole variable
```

Deep Dive : add vs update

- `add` — single hashable item
- `update` — iterable spread (like `extend` for lists)
- `update` accepts strings — spreads characters

- update dict spreads keys (or .keys())
- duplicates ignored silently
- frozenset-in-set allowed

PYTHON

```
S = {1,2,3,4}
# add
# S.add(5)
# print(S)
# update
S.update([5,6,7])
print(S)
```

PYTHON

```
S = {1,2,3,4}
S.add(5)           # one element → {1,2,3,4,5}
S.update([5,6,7,8]) # multiple elements (any iterable)
S.add([5])         # ❌ List unhashable
S.add((5,))        # ok tuple hashable
```

Deep Dive : Adding & removing set entries

Sets are mutable, but there is no "position" — you add and remove by **value**.

- `add` takes one hashable item; `update` spreads an iterable in, like `extend` does for lists.
- The **remove vs discard** difference is the interview favourite: `remove` is strict and raises `KeyError` when the value is missing; `discard` is tolerant and silently does nothing.
- Because sets are unordered, `pop()` has no "last element" — it returns an arbitrary member.
- Duplicates are ignored silently: adding an existing value leaves the set unchanged.

PYTHON

```
S = {1, 2, 3}
S.add(4)           # add a single hashable item -> {1,2,3,4}
S.update([5, 6])   # add many items from any iterable
S.discard(9)       # remove by value; missing -> NO error
S.remove(9)        # remove by value; missing -> KeyError!
S.pop()            # removes and returns SOME (arbitrary) element
S.clear()          # empty the set -> set()
del S              # delete the whole variable
```

Deep Dive : Deleting from sets

- Set items no index → delete by VALUE
- `remove` strict, `discard` tolerant — pick wisely
- `pop()` — unpredictable order
- Interview: remove vs discard difference

- No indexing → no positional delete
- del on set slice also impossible

PYTHON

```
# del
s = {1,2,3,4,5}
# print(s)
# del s[0]
# print(s)
# discard
# s.discard(50)
# print(s)
# remove
# s.remove(50)
# print(s)
# pop
# s.pop()
# clear
s.clear()
print(s)
```

PYTHON

```
s = {1,2,3,4,5}
s.remove(3)      # by VALUE; missing → KeyError
s.discard(3)     # by VALUE; missing → NO error
s.pop()          # removes & returns arbitrary element
s.clear()        # empty set
del s            # delete whole variable
```

Theory : Set operators - the four main operations

Membership & iteration work as always : `1 not in s1` , `for i in s1` .

These operators are the reason sets exist - deduplication and "is it common?" answers in one line.

PYTHON

```
s1 = {1, 2, 3, 4, 5}
s2 = {4, 5, 6, 7, 8}

s1 | s2      # UNION      -> {1,2,3,4,5,6,7,8}   (everything)
s1 & s2      # INTERSECTION -> {4,5}           (in BOTH)
s1 - s2      # DIFFERENCE  -> {1,2,3}          (in s1, not s2)
s1 ^ s2      # SYMMETRIC DIFF. -> {1,2,3,6,7,8} (in one, not both)
```

Deep Dive : Set operations — membership & iteration

- **Membership** is the fastest thing a set does: `1 not in s1` → `False` , `1 in s1` → `True` , all in O(1) thanks to the hash table.

- **Iteration** walks the set in some internal (hash) order, which is not the insertion order and is not guaranteed: `for i in s1: print(i)` prints the members but you should never rely on the order.
- Because there is no position, you cannot slice a set (`s1[0:3]` → `TypeError`), and you cannot ask "what is element number 2?" — you can only test membership or iterate the whole set.
- These two operations (membership + iteration) are exactly what the code cell below demonstrates; the actual union/intersection/difference operator examples follow in the next deep-dive cell.

PYTHON

```
s1 = {1, 2, 3, 4, 5}
s2 = {4, 5, 6, 7, 8}
```

Deep Dive : Set operations — union/intersection/difference

OP	SYMBOL	MEANING
Union		elements in either
Intersection	&	in both
Difference	-	in first, not second
Sym diff	^	in exactly one

- Non-mutating — new set returned
- Mutating twins: `update/intersectionupdate/differenceupdate/symmetricdifferenceupdate`
- Same via methods: `.union()` `.intersection()` `.difference()`
- Venn diagram mental model
- Operators only for sets; methods accept any iterable

PYTHON

```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}
s1 | s2
# Union(|)
s1 & s2
# Intersection(&)
s1 - s2
# Difference(-)
s2 - s1
# Symmetric Difference(^)
s1 ^ s2
# Membership Test
1 not in s1
# Iteration
for i in s1:
    print(i)
```

PYTHON

```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}
s1 | s2      # Union      → {1,2,3,4,5,6,7,8}
s1 & s2      # Intersection → {4,5}
s1 - s2      # Difference  → {1,2,3} (in s1 not s2)
s1 ^ s2      # Symmetric diff → {1,2,3,6,7,8}
```

Theory : Set methods - the “new vs in-place” pattern

For every operator there is a method pair. One RETURNS a new set, the `*_update` version mutates it :

Comparison methods (return bool) : Rule of thumb : prefix `update` versions when you want to MODIFY the original, plain versions when you want to keep both sets and just inspect.

PYTHON

```
# Len/sum/min/max/sorted
s = {3,1,4,5,2,7}
len(s)
sum(s)
min(s)
max(s)
sorted(s, reverse=True)
```

PYTHON

```
# union/update
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}

# s1 | s2
s1.union(s2)

s1.update(s2)
print(s1)
print(s2)
```

Frozenset

Frozen set is just an immutable version of a Python set object

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Deep Dive : frozenset

- frozenset = immutable set
- All read ops (union, intersection, membership, len, iteration) work
- No add/remove/update — immutable

- Hashable → set elements, dict keys, nested
- 2D sets possible only with frozensets:
- Use for constants, config-like collections
- created: `frozenset(iterable)`

PYTHON

```
# create frozenset
fs1 = frozenset([1,2,3])
fs2 = frozenset([3,4,5])

fs1 | fs2
```

PYTHON

```
# what works and what does not
# works -> all read functions
# doesn't work -> write operations
```

Theory : frozenset & set comprehension

frozenset - the immutable set : Because a frozenset is hashable, you can nest them -> "2D sets" : Use frozenset when a set must itself live inside a set or as a dict key.

Set comprehension - same shape as list comprehension, with {} : Order in the output is NOT guaranteed - it is a set after all.

PYTHON

```
fs1 = frozenset([1, 2, 3])
fs1 | frozenset([3,4,5])    # works - read operations are allowed
fs1.add(6)                  # TypeError - write operations are forbidden
```

PYTHON

```
fs = frozenset([1, 2, frozenset([3, 4])]) # Legal!
```

Deep Dive : Set comprehension

- Same shape as list comp but `{}` → set
- Duplicates auto-dropped — unique values
- Order random (sets unordered)
- `{x for x in ...}` vs `{i:i**2 for i in ...}` → dict vs set
- Comprehension main forms:
- Useful for unique filtering
- Remember: `{}` alone → empty DICT

PYTHON

examples

```
{i**2 for i in range(1,11) if i>5}
```

PYTHON

```
{i**2 for i in range(1,11) if i>5}
```

{36, 49, 64, 81, 100} – squares of 6..10

Dictionary

Dictionary in Python is a collection of keys values, used to store data values like a map, which, unlike other data types which hold only a single value as an element.

In some languages it is known as map or associative arrays.

```
dict = { 'name' : 'nitish' , 'age' : 33 , 'gender' : 'male' }
```

Characterstics:

- Mutable
- Indexing has no meaning
- keys can't be duplicated
- keys can't be mutable items

Create Dictionary

Dictionaries

Key-value mappings, unordered in older versions, insertion ordered from 3.7+. Keys must be hashable.

When to use

Lookup by key, structured data.

Theory : Dictionaries - key:value maps (creation)

A dict stores data as **key** -> **value** pairs, like a phone book or a map.

Three hard rules :

1. **Keys must be unique** - repeat a key and the LAST value silently wins : `{'name':'nitish','name':'rahul'}`
-> `{'name': 'rahul'}`.
2. **Keys must be immutable (hashable)** - strings, numbers, tuples are keys; `[1,2]` is not.
3. Values can be ANYTHING - mutable, nested, whatever.

PYTHON

```
d = {} # empty dict
d1 = {'name': 'nitish', 'gender': 'male'}
d2 = {(1,2,3): 1, 'hello': 'world'} # mixed keys - tuple keys are fine
s = { # 2D / nested dict - JSON shape!
    'name': 'nitish',
    'subjects': {'dsa': 50, 'maths': 67}
}
d4 = dict([('name', 'nitish'), ('age', 32)]) # constructor from pairs
```

Deep Dive : Creating dictionaries

- Stores **key:value** maps — key unique, hashable
- Values any type; keys must be hashable (str/int/tuple)
- Empty BOTH: `{}` dict, `set()` set (trap)
- Modern insertion order preserved (py3.7+)
- Access fast $O(1)$ — hash table
- Nested dicts common (JSON-like):

PYTHON

```

# empty dictionary
d = {}
d
# 1D dictionary
d1 = { 'name' : 'nitish' , 'gender' : 'male' }
d1
# with mixed keys
d2 = {(1,2,3):1, 'hello':'world'}
d2
# 2D dictionary -> JSON
s = {
    'name':'nitish',
    'college':'bit',
    'sem':4,
    'subjects':{
        'dsa':50,
        'maths':67,
        'english':34
    }
}
s
# using sequence and dict function
d4 = dict([('name', 'nitish'), ('age', 32), (3, 3)])
d4
# duplicate keys
d5 = {'name':'nitish', 'name':'rahul'}
d5
# mutable items as keys
d6 = {'name':'nitish', (1,2,3):2}
print(d6)

```

PYTHON

```

d = {} # empty dict
d1 = {'name':'nitish', 'age':32}
dict([('a',1), ('b',2)]) # from pairs
dict(zip(keys, vals)) # from zip
dict(name='nitish', age=32) # keyword

```

Theory : Reading values - [] vs get()

- `[key]` is direct and fast, but throws **KeyError** on a missing key.
- `.get(key, default)` NEVER throws - perfect when the key may be absent.
- **Nested access** chains the brackets : `s['subjects']['maths']`.

Golden rule : if there is ANY chance the key is missing, use `get()` (or check `key in d` first).

PYTHON

```
my_dict = {'name': 'Jack', 'age': 26}
my_dict['age']      # 26
my_dict['xxx']      # KeyError: 'xxx' (missing key CRASHES)
my_dict.get('age')  # 26 (safe)
my_dict.get('xxx')  # None (no crash!)
my_dict.get('xxx', 0) # 0 (custom default)
```

Deep Dive : [] vs get for reading

- `d[key]` — strict; missing → `KeyError`
- `d.get(key, default)` — safe; default if none
- get default param
- `d.setdefault` — gets + sets if missing
- `key in d` — membership check before access
- Often used in loops/validator code
- `values()` vs `keys()` vs `items()` covered next

PYTHON

```
my_dict = {'name': 'Jack', 'age': 26}
# []
my_dict['age']
# get
my_dict.get('age')

s['subjects']['maths']
```

PYTHON

```
my_dict = {'name': 'Jack', 'age': 26}
my_dict['age']      # 26
my_dict['city']      # ❌ KeyError
my_dict.get('city')  # None (safe!)
my_dict.get('city', 'NA') # 'NA' default
```

Theory : Adding, editing & removing entries

Insert or overwrite - assignment does both :

Removing :

Nested dicts work the same : Remember : the SAME bracket-assignment both inserts new pairs and edits existing ones - there is no separate "add" method.

PYTHON

```
d4['gender'] = 'male'
d4
d4['weight'] = 72
d4

s['subjects']['ds'] = 75
s
```

PYTHON

```
d4['gender'] = 'male'    # adds if new
d4['weight'] = 72        # adds
d4['age'] = 33           # EDITS if the key already exists
```

Remove key-value pair

PYTHON

```
d = {'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male', 'weight': 72}
# pop
#d.pop(3)
#print(d)
# popitem
#d.popitem()
# d.popitem()
# print(d)
# del
#del d['name']
#print(d)
# clear
d.clear()
print(d)

del s['subjects']['maths']
s
```

Editing key-value pair

PYTHON

```
s['subjects']['dsa'] = 80
s
```

Dictionary Operations

- Membership
- Iteration

Dictionaries

Key-value mappings, unordered in older versions, insertion ordered from 3.7+. Keys must be hashable.

When to use

Lookup by key, structured data.

Theory : Membership & iteration on dicts

Membership on a dictionary always asks "is this a KEY?", not "is this a value?".

Iteration : The for-loop over a dict yields keys by default - reach the value with `d[k]`.

PYTHON

```
print(s)

'name' in s
```

PYTHON

```
d = {'name': 'nitish', 'gender': 'male', 'age': 33}

for i in d:
    print(i, d[i])
```

Theory : dict functions - len, sorted, views, update

sorted / max / min all operate on **keys**.

Views : `d.items()`, `d.keys()`, `d.values()` are live views - they show the dict's current contents (`list()` them to snapshot).

Merging - update() : update merges the second dict INTO the first, later values winning on duplicates.

PYTHON

```
len(d)          # number of pairs
sorted(d)       # keys sorted -> a LIST (reverse=True for descending)
max(d)          # Largest KEY, alphabetically/numerically
```

PYTHON

```
d1 = {1:2, 3:4, 4:5}
d2 = {4:7, 6:8}
d1.update(d2)    # d1 absorbs d2; shared key 4 is OVERWRITTEN -> {1:2, 3:4, 4:7, 6:8}
```

Deep Dive : dict functions

- len — key count
- sorted(d) sorts by KEYS; returns list
- max/min compare keys
- items/keys/values — view objects, behaves iterable
- update — merge/overwrite

- dict → JSON directly with `json.dumps`
- `copy()` / `fromkeys()` — factory

PYTHON

```
# Len/sorted
len(d)
print(d)
sorted(d, reverse=True)
max(d)
```

PYTHON

```
# items/keys/values
print(d)

print(d.items())
print(d.keys())
print(d.values())
```

Dictionary Comprehension

{ key: value for vars in iterable }

Theory : Dictionary comprehension - five patterns

Same skeleton as list comprehension, but the expression is a `key:value` pair :

Patterns 2-5 cover most real needs : renaming values, mapping keys to values, dropping bad entries, building structured tables.

PYTHON

```
# 1. Basic : squares of 1..10
{i: i*i for i in range(1, 11)}

# 2. Transform an EXISTING dict (.items())
distances = {'delhi': 1000, 'mumbai': 2000}
{key: value*0.62 for key, value in distances.items()}

# 3. Pair two lists with zip()
{i: j for i, j in zip(days, temp_C)}

# 4. Filter with if
{key: value for key, value in products.items() if value > 0}

# 5. Nested - multiplication tables 2 to 4
{i: {j: i*j for j in range(1, 11)} for i in range(2, 5)}
```

Deep Dive : Dict comprehension — five patterns

Five patterns:

1. Build from iterable of keys
2. From existing dict (items)
3. From keys list
4. From zip
5. Filtering with if

Nested:

- Same skeleton: `{key_expr: value_expr for item in iterable if cond}`
- Common: data → lookup maps
- Read as english: for each ... produce ...
- vs set comp `{}` — presence of colon = dict

PYTHON

```
# print 1st 10 numbers and their squares
{i:i**2 for i in range(1,11)}
```

PYTHON

```
distances = {'delhi':1000,'mumbai':2000,'bangalore':3000}
print(distances.items())
```

Functions in Python

Theory : Anatomy of a function — def, docstring, return

- `def` starts the definition; the **name** follows, then `(parameters)` and a colon.
- The **body must be indented** (usually 4 spaces) — like `if` / `for`, the indentation is the block in Python.
- The **docstring** (triple quotes, right after the `def` line) documents the purpose, inputs, and output. It is optional but strongly recommended; `help(f)` and `f.__doc__` show it later.
- `return` hands a value back to the caller. Without a `return`, the function returns `None`.
- Later you **call** it: `name_of_function(arg1, arg2)` — the arguments fill the parameters in order.

Functions exist so you write a tricky logic ONCE and reuse it everywhere — the whole point of this session.

PYTHON

```
def name_of_function(parameter1, parameter2):
    """docstring – explains what this function does"""
    # body – indented lines
    return something
```

PYTHON

```
def is_even(num):
    """input - an integer ; output - 'even' / 'odd'"""
    if type(num) == int:
        return 'even' if num % 2 == 0 else 'odd'
    return 'not an integer'
```

Deep Dive : Anatomy of a function

Parts:

1. `def` — keyword
 2. `is_even` — function name (identifier)
 3. `(num)` — parameters (inputs)
 4. `"""docstring"""` — documentation (optional)
 5. body — indented lines
 6. `return` — output value
- Call: `is_even(4)` — Even
 - Builtin functions have **doc** too: `print(len.__doc__)`
 - Function name → first-class (object)
 - Parameters vs arguments: parameters = definition side; arguments = call side
 - default returns None if no return
 - Docstring = `help()` access

PYTHON

```
def is_even(num):
    """
    This function returns if a given number is odd or even
    input - any valid integer
    output - odd/even
    created on - 16th Nov 2022
    """
    if type(num) == int:
        if num % 2 == 0:
            return 'even'
        else:
            return 'odd'
    else:
        return 'pagal hai kya?'
```

PYTHON

```
# function
# function_name(input)
for i in range(1,11):
    x = is_even(i)
    print(x)
```

Theory : Two points of view - author vs user

EVERY function can be seen from two sides :

1. The AUTHOR (definer)

- Decides what inputs it needs (parameters) and what it computes.
- Writes the docstring so others understand it.
- Controls the return value.

2. The USER (caller)

- Only cares : "give me inputs, get back the result".
- Reads `type.__doc__`, `print.__doc__` to LEARN a built-in they did not write.
- Calling with a bad input should give a helpful error, not a crash.

Parameters vs Arguments - the one-line difference : Parameters are the empty boxes in the recipe; arguments are the ingredients you drop in.

PYTHON

```
def power(a, b): # a, b = PARAMETERS (names, at definition time)
    return a ** b
power(2, 3) # 2, 3 = ARGUMENTS (values, at call time)
```

Deep Dive : Author vs User POV

Two perspectives: **Author** (who writes function)

- Defines parameters, logic, return
- expects types reasonable
- docstring explains contract
- Validate/type-hint inputs

User (who calls)

- Only needs name, inputs, output
- Doesn't need internal implementation
- Reads docstring/help
- Must pass right types — else errors

- Function = abstraction: interface (what) vs implementation (how)
- Similar to library: user calls `print/math.sqrt` without knowing internals
- Contracts: input type assumptions — Python dynamic, no compile check → runtime errors

PYTHON

```
is_even('hello')
```

PYTHON

```
is_even('hello') # ❌ TypeError: % not supported for str and int
```

Types of Arguments

- Default Argument
- Positional Argument
- Keyword Argument

Theory : Default, Positional & Keyword arguments

1. **Default values** - if the caller passes nothing, the default is used :
2. **Positional arguments** - matched by ORDER :
3. **Keyword arguments** - matched by NAME, order free :

Rules :

- Positional first, keywords after : `power(2, b=3)` OK, `power(a=2, 3)` -> `SyntaxError`.
- Once you use `=`, every later argument must use `=` too.
- You may mix : `power(2, b=3)`.
- Missing a required (non-default) parameter -> `TypeError`.

PYTHON

```
def power(a=1, b=1):
    return a**b
```

PYTHON

```
power()
```

Deep Dive : Default, positional & keyword args

- **Default argument** — optional; if not passed, default used
- **Positional** — order maps to parameters
- **Keyword** — name=value; order flexible
- Rules:
 - positional first, then keyword: `power(2, b=3)` ✓

- no positional after keyword: `power(a=2, 3)` X
- defaults must come after non-default params
- Defaults evaluated once at def time (immutable default warning)
- Mixed ordering is common gotcha

PYTHON

```
# positional argument
power(2,3)
```

PYTHON

```
# keyword argument
power(b=3, a=2)
```

`*args` and `**kwargs`

`*args` and `**kwargs` are special Python keywords that are used to pass the variable length of arguments to a function

Theory : `*args` & `**kwargs` - variable-length arguments

Sometimes you don't know how many arguments a call will pass. The stars handle that.

Points to remember :

1. **Order matters** : normal arguments -> `*args` -> `**kwargs` . Nothing can follow `**kwargs` .
2. `args` / `kwargs` are only a **convention** - the notebook proves it with `*kwargs` and `**salman` . The `*` is what does the magic.
3. `*args` = tuple of positional extras; `**kwargs` = dict of keyword extras.
4. Use them for flexible APIs, wrappers, decorators.

PYTHON

```
# *args
# allows us to pass a variable number of non-keyword arguments to a function.

def multiply(*kwargs):
    product = 1

    for i in kwargs:
        product = product * i

    print(kwargs)
    return product
```

PYTHON

```
def multiply(*kwargs):    # *args collects EXTRAS into a tuple
    product = 1
    for i in kwargs:
        product *= i
    return product
multiply(1,2,3,4,5,6,7,8,9,10,12)    # any count, no list needed!
```

Deep Dive : *args & **kwargs

- `*args` — collects VARIABLE NUMBER of positional args into TUPLE
- `**kwargs` — collects keyword args into DICT
- Name `args` / `kwargs` convention (any name works)
- Order: `def f(a, b, *args, **kwargs)`
- Required before `*args`; `**kwargs` last
- Unpack inverse: `f(*L)`, `f(**d)` — spread into call
- Real use: wrappers, decorators, flexible libs
- Empty allowed: `*args=()`, `**kwargs={}`

PYTHON

```
multiply(1,2,3,4,5,6,7,8,9,10,12)
```

PYTHON

```
# **kwargs
# **kwargs allows us to pass any number of keyword arguments.
# Keyword arguments mean that they contain a key-value pair, like a Python dictionary.

def display(**salman):

    for (key,value) in salman.items():
        print(key, '->', value)
```

Points to remember while using `*args` and `**kwargs`

- order of the arguments matter(normal -> `*args` -> `**kwargs`)
- The words “args” and “kwargs” are only a convention, you can use any name of your choice

Theory : return & None - the output channel

A function without `return` still returns something - the special value `None` : `None` means "no value". `print` shows it, and code expecting a value gets confused by it - a classic bug source.

What return does :

1. Ends the function immediately (code after it is skipped).
2. Hands a value back to the caller : `z = f(x)` .

3. A function may have many `return` statements (one per branch).
4. With no `return` at all, the function still finishes by returning `None`.

Rules for clean functions : if the function COMPUTES something, return it; if it does an ACTION (print, modify), returning `None` is normal.

PYTHON

```
L = [1, 2, 3]
print(L.append(4))  # None - append changes the list but returns nothing
```

Deep Dive : return & None

- No return → function returns **None**
- `print(L.append(4))` → `None` (method mutates, no return)
- Side-effect functions (like `append`) vs pure (return)
- `return` without value → `None`
- `None` in bool context → falsy
- Implicit `None` — classic interview point
- Mutation vs return — know which methods behave how

PYTHON

```
L = [1, 2, 3]
print(L.append(4))
print(L)
```

PYTHON

```
def f(x):
    x = x + 1  # no return statement
print(f(5))  # None - implicit return
L = [1, 2, 3]
print(L.append(4))  # None - append mutates, returns None
```

Variable Scope

What/Why

Variables reference objects in memory. Data types determine operations allowed and behavior.

Semantics

Python is dynamically typed. Assignment binds names to objects; mutability matters.

Common mistakes

Confusing assignment vs comparison, mutability side effects.

Theory : Scope - global vs local

A name is either known everywhere (global) or only inside a function (local).

Local shadows global :

The trap - UnboundLocalError : Assignment ANYWHERE in the function makes Python treat the name as local, even if you only meant to read the global.

Passing values in : numbers/strings are passed by value - the function gets a copy (`x = x + 1` inside does NOT move the outer x). To get a result out, there is `return` .

PYTHON

```
x = 5                                # GLOBAL x
def g(y):
    print(x)                        # reads global x - allowed
    print(x + 1)
g(x)
print(x)                            # -> 5, 6, 5
```

PYTHON

```
x = 5
def f(y):
    x = 1                          # a LOCAL x is created - the global is untouched
    x += 1
    print(x)                        # 2
f(x)
print(x)                            # 5
```

Deep Dive : Scope — global vs local variables

Rules:

- Variables created inside function = **local**
- Top-level = **global** — visible inside (read), not writable:
- To write global: `global x`
- Parameter y also local
- LEGB: Local → Enclosing → Global → Builtin
- Read allowed; assignment creates local
- Integers immutable — passing copies reference

PYTHON

```
def g(y):
    print(x)
    print(x+1)
x = 5
g(x)
print(x)
```

PYTHON

```
def f(y):
    x = 1
    x += 1
    print(x)
x = 5
f(x)
print(x)
```

Nested Functions

Functions

Encapsulate reusable logic (DRY, modularity). Parameters/return, scope, docstrings.

Theory : Nested functions + the recursion danger

A function defined INSIDE another function :

Nested scoping - inner vs outer names : Resolution order : local -> enclosing -> global -> built-in (LEGB).

Recursion + the crash course : Every recursive call needs a **base case** that stops the chain, or recursion never ends.

PYTHON

```
def f():
    def g():
        print('inside g')
    g()          # f calls its inner function
    print('inside f')
```

PYTHON

```
def g(x):
    def h():
        x = 'abc'      # h declares its OWN local x - g's x is untouched
    x = x + 1          # g's own x
    return x
```

Deep Dive : Nested functions & recursion danger

- Function defined inside another = nested

- Inner can reference outer scope (closures)
- Danger: recursion without base case → RecursionError (stack depth)
- Every call pushes a stack frame (~1000 default)
- Recursion needs termination condition
- g as inner helper — encapsulation
- To modify outer var: `nonlocal`
- Nested gives scoping but over-recursion risk

PYTHON

```
def f():
    def g():
        print('inside function g')
        f()
    g()
    print('inside function f')
```

PYTHON

```
f()
```

Functions are 1st class citizens

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Theory : Functions are objects - first-class citizens

In Python a function is a value, just like an int or string. Proof :

Return a function from a function :

Pass a function as an argument : Because functions flow like any other object, we can build higher-order tools (map, filter, reduce) - next section!

PYTHON

```
# type and id
def square(num):
    return num**2

type(square)

id(square)
```

PYTHON

```

type(square)      # function
id(square)         # it has an address in memory

x = square         # 1. REASSIGN to another name
x(3)               # call it -> 9

del square         # 2. DELETE it
square(3)          # NameError - it is gone!

L = [1, 2, square] # 3. STORE it in containers
L[-1](9)           # call through the list

s = {square}       # even sets accept functions (they are hashable)

```

Deep Dive : Functions are first-class objects

First-class citizen = function can be:

1. assigned to variable
 2. passed as argument
 3. returned from function
 4. stored in data structures (list, set, dict)
 5. have attributes
- This powers map/filter/reduce, decorators, callbacks
 - cfg: function name = reference to object

PYTHON

```

# reassign
x = square
id(x)
x(3)

```

PYTHON

```

a = 2
b = a
b

```

Benefits of using a Function

- Code Modularity
- Code Readability
- Code Reusability

Lambda Functions

A lambda function is a **small anonymous function** — it has no name and is written in one line.

A lambda can take any number of arguments, but its body can contain only **one expression**. The value of that expression is automatically returned.

Lambdas are most useful when passed inline to higher-order functions like `map`, `filter`, and `sorted` — you will see exactly that pattern in the upcoming cells. The next cell explains the full syntax and when to prefer a lambda over a normal `def`.

PYTHON

```
lambda x: x**2           # the function itself (no name)
(lambda x: x**2)(5)      # call it straight away -> 25
```

Theory : Lambda - the small anonymous function

A lambda is a function with NO name and only ONE expression :

Lambda vs a normal def :

DEF	LAMBDA
has a name	anonymous (used on the spot)
many statements	ONE expression only
needs return	the expression IS the return value
reusable everywhere	usually used inline once

Where lambdas shine - tiny one-liners : By themselves lambdas feel weak - their power appears when passed INTO higher order functions.

PYTHON

```
lambda x: x**2           # the function itself
a = lambda x: x**2       # assign it a name, then
a(5)                     # -> 25
```

```
add = lambda x, y: x + y
add(5, 2)                # -> 7
```

PYTHON

```
(lambda s: 'a' in s)('hello')      # -> False
(lambda x: 'even' if x % 2 == 0 else 'odd')(6)  # -> 'even' (ternary inside!)
```

Deep Dive : Lambda functions

- `lambda args: expression` — one-liner anonymous
- Expression only: no statements, no explicit return
- Storable, passable, returnable — like `def`
- No name by default → inline use

- Bodies: `lambda s: 'a' in s`, ternary: `lambda x: 'even' if x%2==0 else 'odd'`
- Common with map/filter/sort

	LAMBDA	NORMAL DEF
name	no	yes
body	one expression	multiple statements
return	implicit	explicit
use	simple inline	complex

- Heavier logic → def
- Docs: lambda no docstring

PYTHON

```
# x -> x^2
lambda x: x**2
```

PYTHON

```
# x,y -> x+y
a = lambda x,y: x+y
a(5,2)
```

Diff between lambda vs Normal Function

- No name
- lambda has no return value(infact, returns a function)
- lambda is written in 1 line
- not reusable

Then why use lambda functions?

They are used with HOF

PYTHON

```
# check if a string has 'a'
a = lambda s: 'a' in s
a('hello')
```

PYTHON

```
# odd or even
a = lambda x: 'even' if x%2 == 0 else 'odd'
a(6)
```

Theory : Higher Order Functions (HOF)

A function that TAKES a function and/or RETURNS a function is a higher-order function.

- `transform` does not care WHICH function it receives - it just applies it.
- `square` and `cube` are the same shape, so either plugs in.

The three famous HOFs are coming :

- `map` - apply a function to every item.
- `filter` - keep only items passing a test.
- `reduce` - fold many items into one.

HOFs let you say WHAT to do (transform each item) without writing the loop boilerplate yourself.

PYTHON

```
def square(x): return x**2
def cube(x): return x**3

def transform(f, L):      # f is a FUNCTION parameter
    out = []
    for i in L:
        out.append(f(i))
    print(out)

transform(lambda x: x**3, [1,2,3,4,5])  # -> [1, 8, 27, 64, 125]
```

Deep Dive : Higher Order Functions (HOF)

- HOF = takes function and/or returns function
- map/filter/reduce — the canonical HOFs
- Decorators build on HOF
- callbacks, event handlers
- Functional programming cleats
- Composition functional pipelines
- Basis for the next three topics (map/filter/reduce)

PYTHON

```
# Example

def square(x):
    return x**2

def cube(x):
    return x**3

# HOF
def transform(f,L):
    output = []
    for i in L:
        output.append(f(i))

    print(output)

L = [1,2,3,4,5]

transform(lambda x:x**3,L)
```

PYTHON

```
def apply(fn, x):
    return fn(x)
apply(square, 3)    # 9 - function as ARG

def make_power(n):
    def power(x):
        return x**n
    return power    # function RETURNED
square_3 = make_power(3)
square_3(2)        # 8
```

Map

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

Theory : map(function, iterable)

`map` applies a function to EVERY item of an iterable : `map(f, L) = [f(x) for x in L]`.

Important : `map` returns a LAZY iterator, not a list. Use `list(map(...))` to force it (or loop it). It stops early if the iterable does.

Perfect partner of lambda : the function often has no reason to live beyond this one call, so an inline lambda is ideal.

PYTHON

```
list(map(lambda x: x**2, [1,2,3,4,5])) # -> [1, 4, 9, 16, 25]

L = [1,2,3,4,5]
list(map(lambda x: 'even' if x%2 == 0 else 'odd', L)) # labels

# map over a list of DICTS - extract one field
users = [{'name': 'Rahul', 'gender': 'male'},
          {'name': 'Ankita', 'gender': 'female'}]
list(map(lambda u: u['gender'], users)) # -> ['male', 'female']
```

Deep Dive : map(function, iterable)

- Applies function to EVERY item
- Returns map OBJECT (iterator) — wrap in list()
- Function can be lambda or named
- Multiple iterables: map(f, a, b) → f(x,y)
- Like list comprehension — map is the functional twin
- Lazy — one pass produced on list()
- Example: `map(str, [1,2,3])` → '1','2','3'
- Use for field extraction, transforms, type conversion

PYTHON

```
# square the items of a list
list(map(lambda x: x**2, [1,2,3,4,5]))
```

PYTHON

```
# odd/even labelling of list items
L = [1,2,3,4,5]
list(map(lambda x: 'even' if x%2 == 0 else 'odd', L))
```

Theory : filter(function, iterable)

`filter` keeps only the items for which the function returns `True` :

How to read it :

- The function is a PREDICATE - returns True/False, does not transform.
- Items that pass stay; items that fail are dropped.
- Original order is preserved.
- Returns a lazy iterator -> wrap with `list()` .

map CHANGES every item; filter SELECTS some items. They compose beautifully : `filter` first, then `map` .

PYTHON

```
L = [3,4,5,6,7]
list(filter(lambda x: x > 5, L))    # -> [6, 7]

fruits = ['apple', 'guava', 'cherry']
list(filter(lambda x: x.startswith('a'), fruits))    # -> ['apple']
```

Deep Dive : filter(function, iterable)

- Keeps only items where function returns **True**
- Function must return bool/truthy
- Returns iterator — wrap list()
- `filter(None, L)` — removes falsy values
- Equivalent: `[x for x in L if cond]`
- FUNCTION filters, comprehension readable
- Doesn't transform values — only selects
- Pythonic for whitelist/keep patterns

PYTHON

```
# numbers greater than 5
L = [3,4,5,6,7]

list(filter(lambda x:x>5,L))
```

PYTHON

```
# fetch fruits starting with 'a'
fruits = ['apple', 'guava', 'cherry']

list(filter(lambda x:x.startswith('a'),fruits))
```

Theory : reduce(function, iterable) - folding

`reduce` repeatedly combines items into one final value. From the `functools` module (not built-in like `map/filter`) :

How it works step by step :**Find the maximum :****Notes :**

- The function takes TWO arguments (accumulator, current item).
- An initial value can be passed : `reduce(f, L, start)` (uses start instead of `L[0]`).
- Use it when a whole collection must "collapse" into one result (sum, product, max, join). If it feels forced, a plain loop is fine.

PYTHON

```
import functools
functools.reduce(lambda x, y: x + y, [1,2,3,4,5]) # -> 15
```

PYTHON

```
((((1+2)+3)+4)+5) # the lambda takes the running total + the next item
```

Deep Dive : reduce(function, iterable) — folding

How reduce works:

- Repeatedly applies function to accumulated result + next item
- Single final value
- `sum()`, `max()`, `min()` are reduce special cases
- Needs `import functools` (reduced from builtins)
- Useful: aggregates like product, gcd, concat, min/max
- Readability: prefer sum/max when built-in exists
- Initializer: `reduce(f, iterable, start)`
- Folding/catamorphism functional concept

PYTHON

```
# sum of all item
import functools

functools.reduce(lambda x,y:x+y,[1,2,3,4,5])
```

PYTHON

```
# find min
functools.reduce(lambda x,y:x if x>y else y,[23,11,45,10,1])
```

Performance and Design Decisions

How lists are stored in memory

What/Why

Mutable ordered sequences for heterogeneous collections.

Semantics

Mutable (in-place), dynamic size, indexable/sliceable.

When to use

When order matters and contents change.

Common mistakes

Shallow copies vs deep copies (aliasing).

How Functions are executed in memory?

What/Why

Functions encapsulate reusable logic (DRY, modularity, abstraction). Enable testing and separation of concerns.

Semantics

Parameters, arguments, return values, scope (LEGB), default args, docstrings.

Common mistakes

Mutable default arguments.

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- container semantics and selection
- mutability, aliasing and copying
- comprehensions and iteration
- function signatures, scope and return values

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

THEORY : LISTS VS ARRAYS & HOW MEMORY WORKS

```
L = [1, 2, 3]
print(id(L))      # ONE unique id for the list object itself
print(id(L[0]))   # the SAME id as id(1) - Python caches small ints
```

PYTHON

DEEP DIVE : LISTS VS ARRAYS & MEMORY

```
L = [1,2,3]

print(id(L))
print(id(L[0]))
print(id(L[1]))
print(id(L[2]))
print(id(1))
print(id(2))
print(id(3))
```

PYTHON

DEEP DIVE : LISTS VS ARRAYS & MEMORY

```
L = [1, 2, 3]
print(id(L))      # address of the list object itself
print(id(L[0]))   # address of the integer 1 it points to
```

PYTHON

DEEP DIVE : CHARACTERISTICS OF A LIST

```
L = [1,2,3,1]
L1 = [3,2,1]

L == L1
```

PYTHON

DEEP DIVE : CHARACTERISTICS OF A LIST

```
L = [1,2,3,1]
L1 = [3,2,1]
L == L1      # False - ORDER MATTERS
```

PYTHON

THEORY : CREATING LISTS — EVERY LEGAL WAY

```
[ ] # empty list
[1, 2, 3, 4, 5] # 1D homogeneous
[1, 2, 3, [4, 5]] # 2D (a list inside a list)
[[[1,2],[3,4]],[[5,6],[7,8]]] # 3D
[1, True, 5.6, 5+6j, 'Hello'] # heterogeneous
```

PYTHON

THEORY : CREATING LISTS — EVERY LEGAL WAY

```
list('hello') # -> ['h','e','l','l','o'] (iterates the string)
list(range(5)) # -> [0,1,2,3,4]
list((1,2,3)) # tuple -> list
```

PYTHON

DEEP DIVE : CREATING LISTS

```
# Empty
print([])
# 1D -> Homo
print([1,2,3,4,5])
# 2D
print([1,2,3,[4,5]])
# 3D
print([[[1,2],[3,4]],[[5,6],[7,8]]])
# Hetrogenous
print([1,True,5.6,5+6j,'Hello'])
# Using Type conversion
print(list('hello'))
```

Glossary

Array (C / Java style)

- Fixed size - you decide the capacity BEFORE filling it. Adding one more element when it is full needs a whole new array.

Python List (dynamic array)

- ****Dynamic**** - grows and shrinks automatically as you add/remove items. Python reserves a little extra room and reclaims it as needed.

Heterogeneous

one list can hold an int, a float, a string, a bool and even another list.

How a list is actually stored

A list does NOT copy the numbers inside itself. It keeps a contiguous block of **references (pointers)** to objects that live elsewhere in memory. `id(L)` is the address of the list, `id(L[0])` is the address of the integer object it points to.

By position (index based)

``del L[i]`` - delete the item at index i.

By value (value based)

``L.remove(value)`` - deletes the FIRST occurrence. If the value is absent -> ``ValueError``.

Arithmetic

``+`` concatenates two lists -> a NEW list (originals untouched) : ``[1,2] + [3,4] = [1,2,3,4]``

Membership

``in`` / ``not in`` -> is a value present? Works at ANY depth for nested lists : ``[5,6] in [[5,6],7]`` -> `True`.

Looping

``for i in L`` walks the items in order. For ``[[[1,2],[3,4]],[[5,6],[7,8]]]``, each ``i`` is one 2D sub-list.

Equality

``==`` compares items one by one in order : ``[1,2,3] == [3,2,1]`` -> `False` (order matters).

Advantages of List Comprehension

More time-efficient and space-efficient than equivalent ``for`` loops (the loop runs inside C).

Shape

``[expression for item in iterable if condition]``

Itemwise

use when you only need the value; fast, readable

Indexwise

when you also need the index / want to mutate a position

Order / positions

> indexing, slicing, indexwise loops

No duplicates

> think `set` | **counting** -> think `dict`

Transform each item

> comprehension / map | **User data** -> input().split()

tuples

immutable, ordered records created with parentheses and a comma — convenient for fixed data, functions returning multiple values, and (because they are hashable) dict keys and set elements.

tuple

an immutable list.

set

an unordered collection of unique, hashable items.

Indexing

exactly like lists :

Slicing

also like lists, but the result is a TUPLE :

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

- ☐ What are Lists
- ☐ Array Vs Lists
- ☐ Theory : Lists vs Arrays & how memory works
- ☐ Deep Dive : Lists vs Arrays & memory
- ☐ Characteristics of a List
- ☐ Theory : The 8 characteristics - one by one

- ☐ Deep Dive : Characteristics of a list
- ☐ Creating a List
- ☐ Theory : Creating lists — every legal way
- ☐ Deep Dive : Creating lists
- ☐ Theory : Indexing & Slicing
- ☐ Deep Dive : Indexing & slicing lists
- ☐ Theory : append vs extend vs insert
- ☐ Deep Dive : append vs extend vs insert
- ☐ Theory : Editing items - in place
- ☐ Deep Dive : Editing list items — in place
- ☐ Theory : Deleting from a list
- ☐ Deep Dive : Deleting from list — 4 tools
- ☐ Operations on Lists
- ☐ Theory : Operations on lists
- ☐ Deep Dive : List operations
- ☐ Theory : The list toolkit - functions & methods